# [Dat Nguyen] Toolchain & Cross-Compilation

# 1. What's Inside a Cross-Compiling Toolchain?

## 1.1 What Is a Compiler?

A compiler is a computer program that translates source code written in a high-level programming language into a lower-level language that a computer can execute.

**Example: Writing a C program on Ubuntu**

```
#include <stdio.h>


int main(void)
{
    printf("Hello, Embedded Linux!\n");
    return 0;
}
```

Compile and run the program:

```
gcc hello.c -o hello
./hello
```

Output:

```
Hello, Embedded Linux!
```

GCC stands for **GNU Compiler Collection**. It is an open-source collection of compilers that translates source code from programming languages into machine code or executable files t>

GCC runs on Ubuntu and normally creates programs for an environment compatible with the system it targets by default.

However, in Embedded Linux, we usually have two systems:

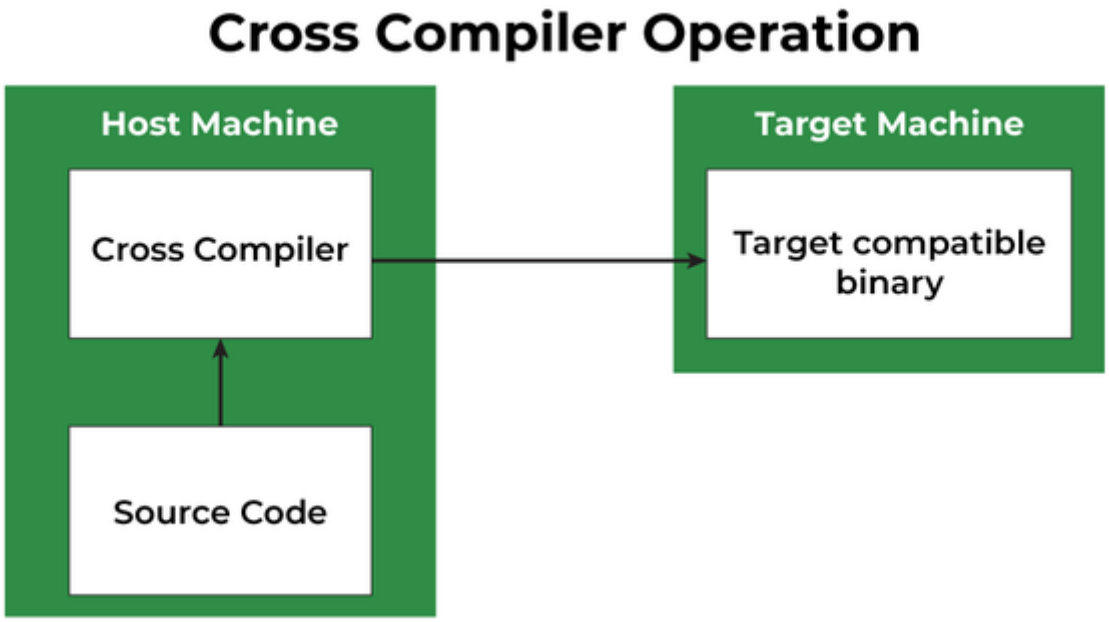
- **Host:** The computer used for software development, such as your Ubuntu laptop.
- **Target:** The device that will run the software, such as a Raspberry Pi, an IoT device, or an embedded board.

If the laptop uses the x86-64 architecture but the program needs to run on ARM64, the laptop's default GCC does not automatically create an ARM64 program.

This is where we need cross-compilation.

# 1.2 What Is Cross-Compilation?

Cross-compilation is the process of compiling a program on one system to create an executable file for a different target system.

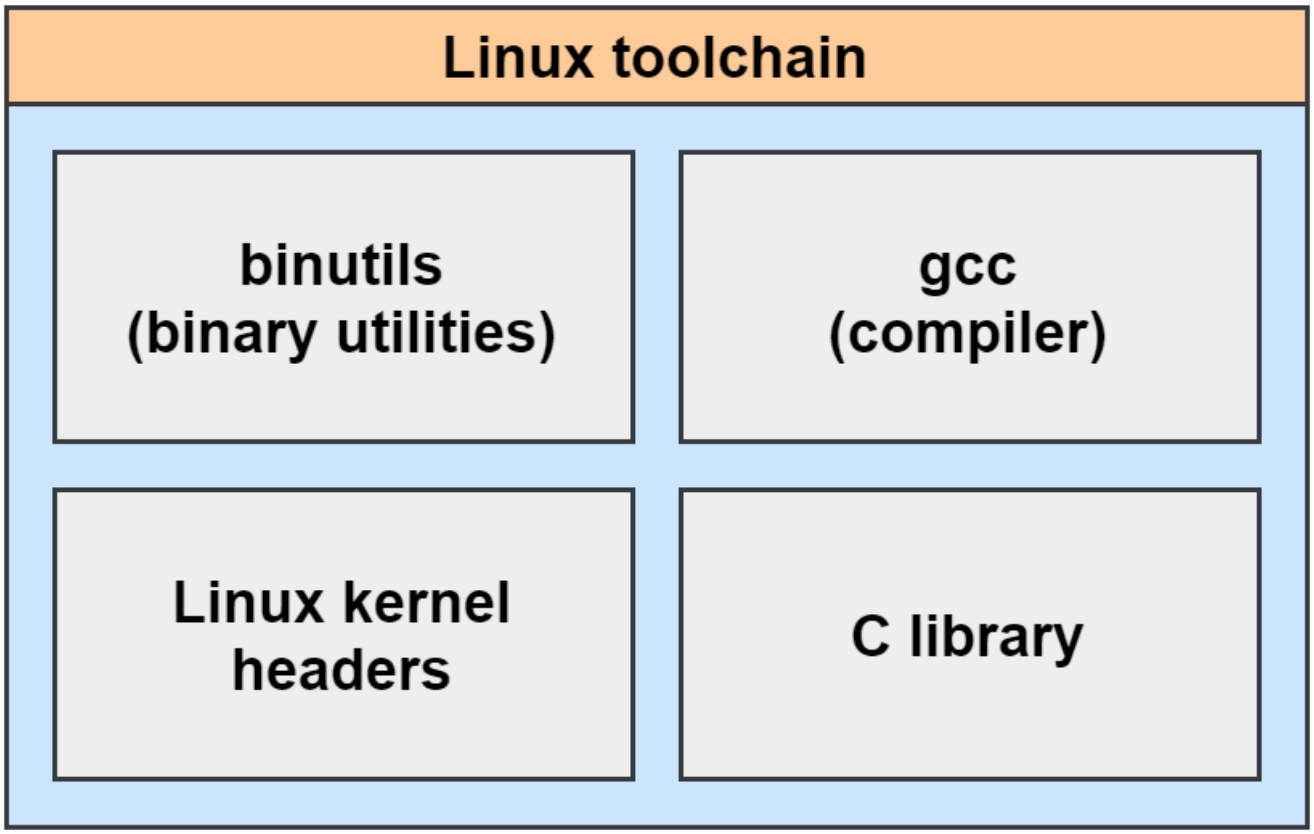



Cross-compilation

The laptop performs the compilation. The Raspberry Pi only needs to receive and run the compiled program. It does not necessarily need GCC installed.

This is especially useful when the target device has a weak CPU, limited RAM, or no complete development environment.

# 1.3 What's Inside a Cross-Compilation Toolchain?




Cross-compilation toolchain components

**GCC Compiler:** This is the core component of the toolchain. It reads C/C++ source code, such as a `.c` file, and translates it into assembly code that matches the target hardware a>

**Binutils (Binary Utilities):** This is a collection of tools for working with binary files. The most important tools include:

- **Assembler (`as`):** Translates assembly code into object files (`.o`).
- **Linker (`ld`):** Links object files and libraries together to create an executable file.
- **Other tools:** `objdump`, `readelf`, and `strip` are used to inspect, analyze, and modify binary files.

**C/C++ Libraries:** The C library is an important component between applications and the kernel. It provides functions that help applications request services from the kernel, and it>

Several C libraries are available. The choice of C library must be made when building the cross-compiling toolchain.

**Kernel Headers:** Kernel headers contain definitions and information that user-space programs need to interact with the Linux kernel, including system-call interfaces and data struc>

**Conclusion:** A complete cross-toolchain combines a compiler, assembler, linker, libraries, and suitable headers for the target system. If a component is missing or incorrectly conf>

---

# 2. Choosing a Target C Library: glibc / uClibc-ng / musl

| Main Feature | glibc | musl libc | uClibc-ng |
|---|---|---|---|
| **Key Characteristics** | A standard C library that provides many functions, supports multiple threads, and offers many features for complex applications. | A C library designed to > | |
| **Common Environments / Applications** | Common Linux distributions for desktop and server systems, such as Ubuntu, and many other Linux systems. | Lightweight environments, such as> | |
| **Main Advantages** | Compatible with a large GNU/Linux software ecosystem; suitable for many applications and third-party software; extensive documentation and a rich ecosystem. | > | |
| **Disadvantages / Limitations** | Uses more storage than some lightweight alternatives; library versions and symbols must be considered when deploying applications to other systems.> | | |

---

# 3. Ready-to-Use Cross-Compiling Toolchains vs. Building a Custom Toolchain

## 3.1 Ready-to-Use Toolchain

A ready-to-use toolchain is a toolchain that has already been built and packaged by someone else.

**Advantages:**

- Quick to install.
- Allows you to start cross-compiling immediately.
- No need to build GCC and the C library from source.
- Suitable for learning, testing, and software development.

**Limitations:**

A prebuilt toolchain is not necessarily optimized or configured for your specific target system.

You need to check:

- Compiler version.
- Target architecture and ABI.
- C library.
- Sysroot.
- Compatibility with the target system.

**Example:**

```
gcc-aarch64-linux-gnu
```

## What Is a Sysroot?

A sysroot is a directory on your development computer that contains the headers and libraries needed to build programs for the target system. It may contain relevant parts of the targ>

A sysroot can be thought of as a copy of the relevant parts of the target system stored on your development computer.

It provides the headers and libraries required for the target architecture and operating environment, helping the cross-compiler build programs that are compatible with the target sys>

When you install a package such as `gcc-aarch64-linux-gnu` on your laptop, the package typically provides access to the target headers and libraries required by the compiler.

The compiler is configured to find the appropriate target files through its sysroot and search paths. When you run the compiler, it can use the ARM64 headers and libraries rather than>

# 3.2 Custom Toolchain

A custom toolchain is a toolchain that you build after selecting and configuring its components.

You can customize:

- Target architecture.
- CPU and related features.
- ABI.
- GCC version.
- Binutils version.
- C library.
- Linux kernel headers.
- Supported build options.

# Comparison: Ready-to-Use vs. Custom Toolchain

| Criterion | Prebuilt Toolchain | Custom Toolchain |
|---|---|---|
| Setup time | Short | Longer |
| Component selection | Limited by the package | Explicitly configurable |
| Learning value | Good for learning cross-compilation | Good for learning how to build a toolchain |
| Reproducibility | Depends on package and version management | Can be improved by saving configuration and source versions |

| Criterion | Prebuilt Toolchain | Custom Toolchain |
| --- | --- | --- |
| Typical use | Application development and quick experiments | Specialized embedded development and controlled build environments |

# 4. Sysroot

A sysroot is a directory that acts as the root of the target system's development environment. It contains the headers and libraries needed for cross-compilation.

Cross-compilation can be difficult if the compiler uses the host system's headers and libraries instead of those intended for the target system. This can lead to compilation errors or>

A sysroot helps solve this problem by providing the target-specific headers and libraries in a separate directory on the host computer.

For example, if your host computer uses x86-64 and your target device uses ARM64, the sysroot provides the appropriate target headers and libraries so that the cross-compiler can buil>

**In short, the sysroot helps the cross-compiler use the correct development files for the target system instead of the host system.**

Revision #2
Created 2026-10-21 07:25:42 UTC by Dat Nguyen
Updated 2026-10-21 07:34:21 UTC by Dat Nguyen